

컴파일러 프로젝트 2025

소프트웨어학과 2021041017 김규현

I. 서론

요즘 대한민국 사회는 너무 날카롭다고 생각한다. 자신과 조금 다르면 이상한 사람을 취급하고 자신보다 뛰어난 사람에 대해 존경심을 표하기보다는 질투심과 허영심, 그 사람의 안 좋은 점을 어떻게든 찾아서 헐뜯기 마련인 예전 시대보다 흥흥해진 사회라고 생각한다. 또한, 코딩 언어들은 영어로 이루어져 있어서 처음 프로그래밍을 배우는 학생들에게는 언어의 장벽이 존재한다. 이런 이유들로 둥글게 말하는 연습이 필요할 것 같았고, 한국어 중에서도 귀여운 말로 프로그래밍 할 수 있는 'Cute Compiler'를 이번 컴파일러 수업의 프로젝트를 기회삼아 만들게 되었다. "고마웁", "요기부터", "저장할랭", "출력해줘" 등과 같은 감사할 줄 알고 귀여운 한국어 표현을 사용하여 진입장벽을 낮추고 둥글둥글 언어에 익숙해지게 하고자 하는 것이 목표여서 제작하게 되었다.

II. 설계 및 구현

C언어와의 비교:

C Language	Cute Language
+ - * /	더하기, 더할랭 빼기, 뺄랭 곱하기, 곱할랭 나누기, 나눌랭
++ --	++ --
+= -= *= /=	+= -= *= /=
if (x > y) { temp = x; }	만약에 x가 y보다 크다면 요기부터 temp에 x를 저장할랭 요기까지 실행할랭
if (x > y) { result = x;	만약에 x가 y보다 크다면 요기부터 result에 x를 저장할랭

<pre> } else { result = y; } </pre>	요기까지 실행할랭 아니라면 요기부터 result에 y를 저장할랭 요기까지 실행할랭
<pre> for (i = 0; i < 5; i++) { temp = x; } </pre>	일단은 i에 0을 저장할랭 i가 5보다 작다면 마지막에 i++ 하면서 요기부터 temp에 x를 저장할랭 요기까지 반복할랭
<pre> while (x < 10) { x++; } </pre>	x가 10보다 작다면 요기부터 x++ 요기까지 반복할랭
<pre> temp = x; x = y; y = temp; </pre>	x와 y의 값을 서로 바꿔칭
<pre> c = (x > y) ? x : y; </pre>	c에 x가 y보다 크다면 x를 저장할랭 아니라면 y를 저장할랭
<pre> printf("%d", x); </pre>	x의 값을 출력할랭
<pre> scanf("%d", &x); </pre>	x의 값을 입력받아칭
<pre> void func { // 함수 내용 } ... func(); // 호출 </pre>	함수이름은 func이고 뭐가들었냐면 요기부터 // 함수 내용 요기까지인 함수를 선언할랭 ... func를 호출할랭
<pre> switch(x) { case1: y = 1; break; case2: y = 2; break; } </pre>	여러 경우가 있엉 x가 1일 경우엔: y에 1을 저장할랭 빠져나가방 2일 경우엔: y에 2를 저장할랭 빠져나가방

컴파일러 구성 요소:

어휘 분석기(Lexical Analyzer): cute.l

구문 분석기(Parser): cute.y

코드 생성기(Code Generator): cute.y 내부의 생성 함수들

심볼 테이블(Symbol Table): 변수와 함수명 관리

어휘 분석(Lexical Analysis) 설계:

토큰 정의.

어휘 분석기는 한국어 키워드와 영어 연산자를 모두 인식한다.

산술 연산자.

+, 더하기, 더할랭 → ADD 토큰

-, 빼기, 뺄랭 → SUB 토큰

*, 곱하기, 곱할랭 → MUL 토큰

/, 나누기, 나눌랭 → DIV 토큰

비교 연산자.

작다면 → LT (Less Than)

크다면 → GT (Greater Than)

작거나 같다면 → LE (Less or Equal)

크거나 같다면 → GE (Greater or Equal)

같다면 → EQ (Equal)

같지 않다면 → NE (Not Equal)

제어 구조.

만약에 → IF

아니라면 → ELSE

요기부터 → LB (Left Brace)

요기까지 → RB (Right Brace)

특수 표현.

저장할랭 → SAVE

실행할랭 → RUN

출력할랭 → PRINTF

입력받아칭 → SCANF

바꿔칭 → SWAP

정규 표현식 패턴:

sp	[\t]	// 공백
nl	\n	// 개행
letter	[A-Za-z]	// 영문자
digit	[0-9]	// 숫자
id	{letter}{letter}{digit}*	// 식별자

LEX 파일 (cute.l):

```
%{
#include <string.h>
#include "cute.h"
#define MAXSYM100
#define MAXSYMLEN 20

#define STMTLIST 500

typedef struct nodeType {
    int token;
    int tokenval;
    struct nodeType *son;
    struct nodeType *brother;
} Node;

char sytbl[MAXSYM][MAXSYMLEN];
int lineno=1;
int insertsym(char *s);
int maxsym=0;
char s[MAXSYMLEN];
```

```

int temp;

Node * MakeNode(int, int);

%}
sp          [ \t]
ws          {sp}+
nl          \n
eletter     [A-Za-z]
hletter     [\xb0-\xfe][\xa0-\xfe]
letter      ({eletter})
digit       [0-9]
id          {letter}({letter}|{digit})*
%%
{ws}        /* do nothing */
{nl}        {lineno++;}
\+|더하기|더할랭      {return(ADD); }
\-|빼기|뺄랭          {return(SUB); }
\=                  {return(ASSGN);}
\=|를              {return(THIS);}
"\*"|곱하기|곱할랭      {return(MUL);}
"/"|나누기|나눌랭      {return(DIV);}
;                  {return(STMTEND);}
\,|부터            {return(LB);}
\,|까지            {return(RB);}
\&|면서            {return(ING);}
\&|만약에          {return(IF);}
\&|가              {return(IS);}
\&|보다            {return(THEN);}
\&|와|과           {return(WA);}
저장할랭          {return(SAVE);}
실행할랭          {return(RUN);}
마지막에          {return(LAST);}
아니라면          {return(ELSE);}
함수이름은        {return(FUNC);}
이고              {return(AND);}
인                {return(IN);}

```

```

선언할랭      {return(INIT);}
뭐가들었냐면  {return(CONTENT);}
함수          {return(FN);}
반복할랭      {return(LOOP);}
작다면        {return(LT);}
크다면        {return(GT);}
"의 값을"      {return(VAL);}
"작거나 같다면" {return(LE);}
"크거나 같다면" {return(GE);}
같다면        {return(EQ);}
"같지 않다면"   {return(NE);}
"+ ="         {return(AA);}
"- ="         {return(SA);}
"* ="         {return(MA);}
"/ ="         {return(DA);}
일단은        {return(FOR);}
"입력받아칭"   {return(SCANF);}
:             {return(COLON);}
"++"          {return(INC);}
"--"          {return(DEC);}
호출할랭      {return(CALL);}
출력할랭      {return(PRINTF);}
이제          {return(NOW);}
"의 값을 서로 바꿔칭" {return(SWAP);}
"일 경우엔"    {return(CASE);}
"빠져나가봐"  {return(BREAK);}
"다 아닌경우엔" {return(DEFAULT);}
"여러 경우가 있영" {return(SWITCH);}
"시작할랭"    {return(START);}
"고마웁"      {return(END); }
func_{letter}+ {return(FNAME);}
{id}          {temp=insertsym(yytext); yylval=MakeNode(ID, temp); return(ID);}
-?{digit}+    {sscanf(yytext, "%d", &temp); yylval=MakeNode(NUM, temp);
return(NUM);}
-?{digit}+₩.{digit}+ {sscanf(yytext, "%f", &temp); yylval=MakeNode(REAL, temp); return(REAL);}
.              {printf("invalid token %s₩n", yytext); }
%%

```

```

int insertsym(char *s)
{
    int i;

    for(i=0;i<maxsym; i++) {
        if(strcmp(s, symtbl[i])==0) return(i);
    }
    if (i<MAXSYM-1) {
        strcpy(symtbl[maxsym],s);
        maxsym++;
        return(maxsym-1);
    }
    else
        {printf("symbol table overflow\n");}
    return(0);
}

yywrap() {}

```

구문 분석(Syntax Analysis) 설계:

문법구조:

BNF(Backus-Naur Form) 형태로 정의된 문법 규칙:

```

program → START stmt_list END
stmt_list → stmt_list stmt | stmt
stmt → assignment | control_flow | function_def | ...

```

추상 구문 트리(AST) 구조:

```

typedef struct nodeType {
    int token;           // 토큰 타입
    int tokenval;        // 토큰 값
    struct nodeType *son; // 자식 노드
    struct nodeType *brother; // 형제 노드
} Node;

```

트리 구성 방식:

son: 해당 노드의 첫 번째 자식

brother: 같은 레벨의 다음 노드

이진트리가 아닌 다진 트리 구조 사용.

AST 순회 및 코드 생성 전략:

“깊이 우선 탐색(DFS)” 알고리즘:

```
void DFSTree(Node *n) {
    if (n == NULL) return;

    // 1. 자식 노드 먼저 처리 (후위 순회)
    DFSTree(n->son);

    // 2. 중첩 구조 분석 (라벨 관리)
    if (n->token == IF) {
        inner_if_cnt = count_if_nodes(n->son);
        if(n->brother != NULL && n->brother->token == ELSE) {
            fprintf(fp, "GOTO ELSELABEL%d\\n", (++ife_cnt));
        }
    }

    // 3. 현재 노드 코드 생성
    prtcode(n->token, n->tokenval);

    // 4. 형제 노드 처리
    DFSTree(n->brother);
}
```

중첩 구조 처리 알고리즘:

문제: 중첩된 제어구조에서 라벨 충돌 방지

해결: 내부 노드 개수 계산을 통한 라벨 오프셋 조정

e.g)

```
int count_if_nodes(Node *n) {
    if (n == NULL) return 0;
    int count = 0;
    if (n->token == IF) count++;
    count += count_if_nodes(n->son);    // 재귀적 탐색
    count += count_if_nodes(n->brother);
    return count;
}
```

메모리 관리 및 할당 전략:

정적 메모리 할당:

```
void dwgen() {
    fprintf(fp, "HALT\n");
    fprintf(fp, "$ -- END OF EXECUTION CODE AND START OF VAR DEFINITIONS --\n");

    // 심볼 테이블 순서대로 메모리 할당
    for(int i = 0; i < maxsym; i++)
        fprintf(fp, "DW %s\n", symtbl[i]);
    fprintf(fp, "END\n");
}
```

임시 변수 관리:

Swap 연산에서의 임시 변수 자동 생성:

```
case SWAPS:
    insertsym("TEMP");           // 임시 변수 자동 등록
    fprintf(fp, "LVALUE TEMP\n");
    break;
```

에러 처리:

구문 에러 복구.

```
stmt_list : stmt_list stmt {$=MakeListTree($1, $2);}
```

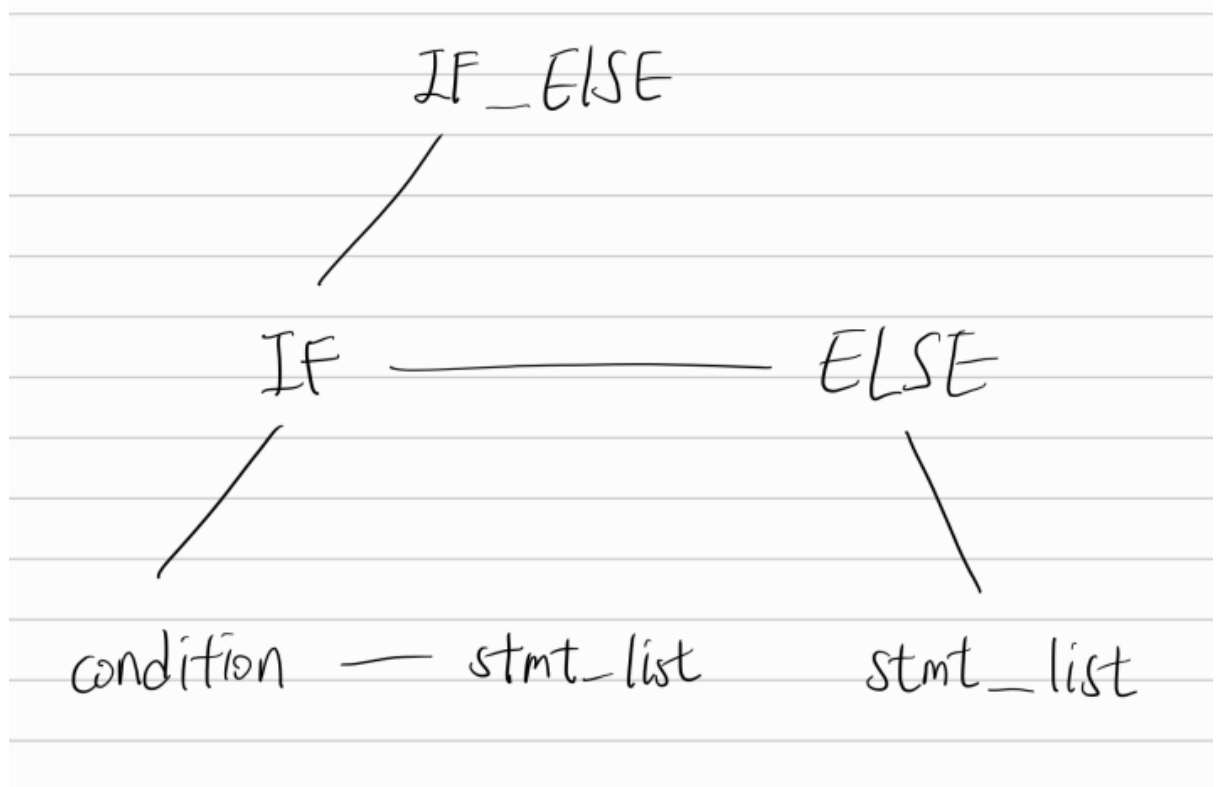
stmt	{ \$=MakeListTree(NULL, \$1); }
error	{ errorcnt++; yyerrok; }

의미 에러 검사.

```
int if_cond_check(int token) {
    if (token == EQ) return IF_EQ;
    if (token == LT) return IF_LT;
    if (token == LE) return IF_LE;
    if (token == GT) return IF_GT;
    if (token == GE) return IF_GE;
    if (token == NE) return IF_NE;
}
```

언어 기능별 설계 상세 서술:

1. if-else 문:



- DFS 탐색 순서에 따른 실행 흐름:
condition → stmt_list → IF → ELSE → stmt_list → IF_ELSE (Post-order (bottom-up)):
- if-else문을 구현하면서 고민했던 점은 조건 분기와 LABEL/GOTO 처리였다. 조건이 참이면 if문의 stmt_list를 실행하고, 거짓이면 else문으로 점프해야 한다. 또한 if문이 완료된 후에는 else문을 건너뛰고 전체 if-else문 밖으로 빠져나가야 한다.
- 조건을 평가한 후, 조건이 참이면 첫 번째 stmt_list를 실행하고, 거짓이면 IF 노드에서 생성한 라벨로 점프하여 ELSE의 stmt_list를 실행한다. IF_ELSE 구문 전체가 완료되면 바로 외부로 빠져나가야 하므로, GOTO문을 통해 지정된 라벨로 이동한다. 이때 GOTO문은 IF 노드에서 해당 라벨을 생성하기 전에 미리 준비되어야 하므로, 다음과 같은 방법을 사용하였다:

```
void DFSTree(Node * n)
{
    Node * brother;
    if (n==NULL) return;

    DFSTree(n->son);
    if (n->token == IF) {
        inner_if_cnt = count_if_nodes(n->son);
        if(n->brother != NULL) {
            if(n->brother->token == ELSE) {
                fprintf(fp, "GOTO ELSELABEL%d\n", (++ife_cnt));
            }
        }
    }

    if(n->token == WHILE)
        inner_w_cnt = count_while_nodes(n->son);

    if(n->token == FOR)
        inner_f_cnt = count_f_nodes(n->son);

    if(n->token == ELSE)
        inner_e_cnt = count_e_nodes(n->son);
}
```

```
prtcde(n->token, n->tokenval);
DFSTree(n->brother);
```

- DFS 코드 생성 함수 내에서 자식 노드 순회를 완료하고 IF 노드로 복귀할 때, 해당 IF 노드의 형제 노드가 ELSE라면 GOTO문을 생성하도록 구현되었다.

```
/* 위 트리를 규칙으로 만든 코드 */
if_stmt      :      IF condition LB stmt_list RB RUN {$2->token=if_cond_check($2-
>token); $$=MakeOPTree(IF, $2, $4); }
              ;
else_stmt    :      ELSE LB stmt_list RB RUN { $$=MakeOPTree(ELSE, $3, NULL); }
              ;

/* stmt 규칙 포함 */
if_stmt else_stmt { $$=MakeOPTree(IF_ELSE, $1, $2); }
if_stmt

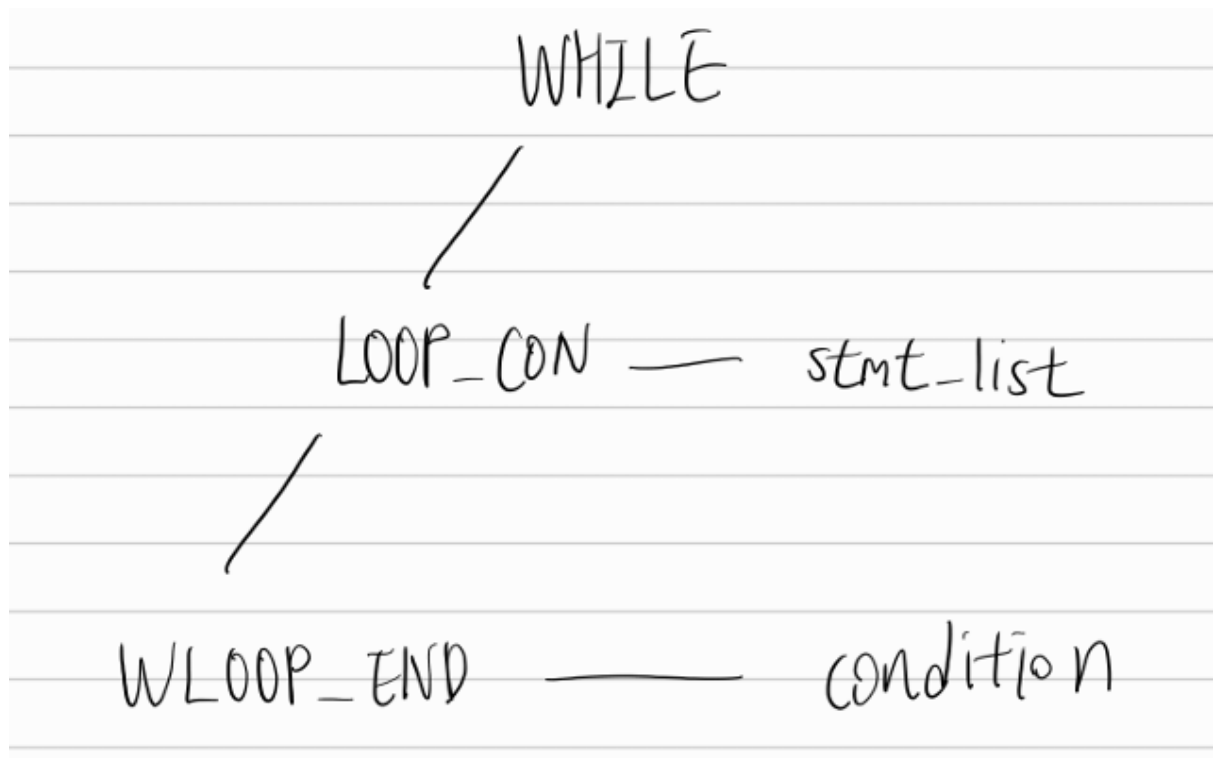
/* 노드에 따라 코드 생성 */
case IF:
    fprintf(fp, "LABEL IFLABEL%d\n", if_cnt - inner_if_cnt);
    break;
case IF_ELSE:
    fprintf(fp, "LABEL ELSELABEL%d\n", ife_cnt-inner_e_cnt);
    break;
case IF_EQ:
    fprintf(fp, "-\n");
    fprintf(fp, "GOTRUE IFLABEL%d\n", ++if_cnt);
    break;
case IF_NE:
    fprintf(fp, "-\n");
    fprintf(fp, "GOFALSE IFLABEL%d\n", ++if_cnt);
    break;
case IF_LT:
    fprintf(fp, "-\n");
    fprintf(fp, "COPY\n");
```

```

        fprintf(fp, "GOPLUS IFLABEL%d\n", ++if_cnt);
        fprintf(fp, "GOFALSE IFLABEL%d\n", if_cnt);
        break;
case IF_GT:
    fprintf(fp, "-\n");
    fprintf(fp, "COPY\n");
    fprintf(fp, "GOMINUS IFLABEL%d\n", ++if_cnt);
    fprintf(fp, "GOFALSE IFLABEL%d\n", if_cnt);
    break;
case IF_LE:
    fprintf(fp, "-\n");
    fprintf(fp, "GOPLUS IFLABEL%d\n", ++if_cnt);
    break;
case IF_GE:
    fprintf(fp, "-\n");
    fprintf(fp, "GOMINUS WLABEL%d\n", ++if_cnt);
    break;

```

2. while 문:



- DFS 탐색 순서에 따른 실행 흐름:
WLOOP_END → condition → LOOP_CON → stmt_list → WHILE
- while문을 시작하면 Wloop_end 노드에서 루프 시작점의 LABEL을 생성하고, condition을 평가한다. 조건이 참이면 stmt_list를 실행한 후 다시 루프 시작점으로 돌아가고, 거짓이면 while문을 빠져나간다. WHILE 노드에서 stmt_list 실행 후 다시 루프 시작점으로 돌아가는 GOTO WLOOP와 루프 종료점 LABEL WLABEL를 생성한다.
- while문을 구현하면서 헤메었던 점은 루프 시작점의 LOOP LABEL을 배치할 적절한 위치가 없다는 것이었다. 이를 해결하기 위해 `wloop\_end`라는 새로운 노드를 생성하여 첫 번째 LOOP LABEL을 출력할 수 있도록 했다.

```

/* 위 트리를 규칙으로 만든 코드 */
loop_con          :          condition wloop_end {$1->token=w_cond_check($1->token);
$$=MakeOPTree(LOOPCON, $2, $1); };

wloop_end        :          { $$=MakeNode(WLOOPEND, WLOOPEND); }

/* stmt 규칙 포함 */
loop_con LB stmt_list RB LOOP    {$$=MakeOPTree(WHILE, $1, $3);}

/* 노드에 따라 코드 생성 */
case WLOOPEND:
    fprintf(fp, "LABEL WLOOP%d\n", ++w_cnt);
    break;

case WHILE:
    fprintf(fp, "GOTO WLOOP%d\n", w_cnt-inner_w_cnt);
    fprintf(fp, "LABEL WLABEL%d\n", w_c_cnt-inner_w_cnt);
    break;

case W_EQ:
    fprintf(fp, "-\n");
    fprintf(fp, "GOTRUE WLABEL%d\n", ++w_c_cnt);
    break;

case W_NE:

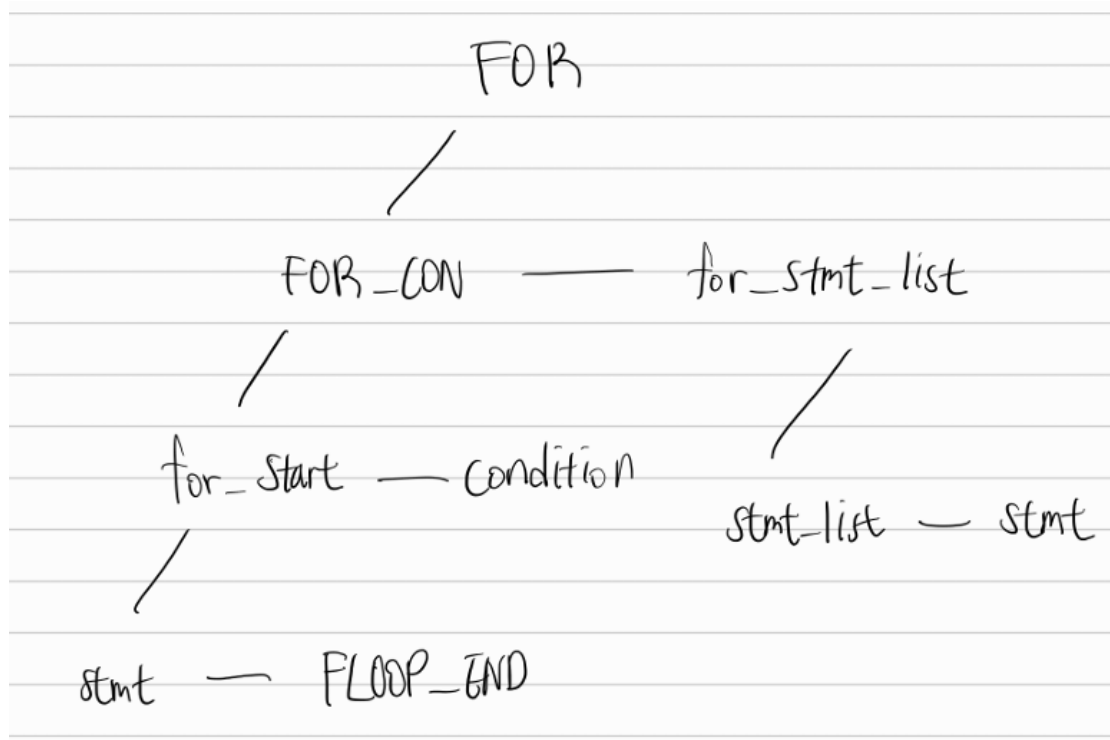
```

```

        fprintf(fp, "-Wn");
        fprintf(fp, "GOFALSE WLABEL%dWn", ++w_c_cnt);
        break;
case W_LT:
        fprintf(fp, "-Wn");
        fprintf(fp, "COPYWn");
        fprintf(fp, "GOPLUS WLABEL%dWn", ++w_c_cnt);
        fprintf(fp, "GOFALSE WLABEL%dWn", w_c_cnt);
        break;
case W_GT:
        fprintf(fp, "-Wn");
        fprintf(fp, "COPYWn");
        fprintf(fp, "GOMINUS WLABEL%dWn", ++w_c_cnt);
        fprintf(fp, "GOFALSE WLABEL%dWn", w_c_cnt);
        break;
case W_LE:
        fprintf(fp, "-Wn");
        fprintf(fp, "GOPLUS WLABEL%dWn", ++w_c_cnt);
        break;
case W_GE:
        fprintf(fp, "-Wn");
        fprintf(fp, "GOMINUS WLABEL%dWn", ++w_c_cnt);
        break;

```

3. for 문:



- for문을 구현하면서 좀 헤메었던 점은 for문의 문법 순서와 DFS에서 생성되는 코드의 순서를 일치시키는 것이었다. while문과 마찬가지로 첫 번째 LOOP LABEL을 배치할 노드가 필요하여 'floop_end'라는 노드를 생성하였다.
- DFS 탐색 순서에 따른 실행 흐름:
 stmt(초기화) → FLOOPEND → condition → FORCON → FORSTART → stmt_list → stmt(증감) → FORSTLIST → FOR
- for문의 문법적 순서(초기화 → 조건 → 내부코드 → 증감)와 DFS 생성 순서가 맞지 않는 문제를 MakeOPTree의 파라미터 인자 순서를 조정하여 해결하였다.

```

/* 위 트리를 규칙으로 만든 코드 */
floop_end          :      { $$=MakeNode(FLOOPEND, FLOOPEND); };

for_stmt_list      :      LAST stmt ING LB stmt_list RB LOOP
                        { $$=MakeOPTree(FORSTLI, $5, $2); }
                        ;

for_con            :      for_start condition      {$2->token=f_cond_check($2->token);

```

```

$$=MakeOPTree(FORCON, $1, $2);}

;

for_start      :      stmt floop_end  {$$=MakeOPTree(FORSTART, $1, $2);}

;

/* stmt 규칙 포함 */
FOR for_con for_stmt_list { $$=MakeOPTree(FOR, $2, $3); }

/* 노드에 따라 코드 생성 */
case F_EQ:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOTRUE FLABEL%dWn", ++f_c_cnt);
    break;
case F_NE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOFALSE FLABEL%dWn", ++f_c_cnt);
    break;
case F_LT:
    fprintf(fp, "-Wn");
    fprintf(fp, "COPYWn");
    fprintf(fp, "GOPLUS FLABEL%dWn", ++f_c_cnt);
    fprintf(fp, "GOFALSE FLABEL%dWn", f_c_cnt);
    break;
case F_GT:
    fprintf(fp, "-Wn");
    fprintf(fp, "COPYWn");
    fprintf(fp, "GOMINUS FLABEL%dWn", ++f_c_cnt);
    fprintf(fp, "GOFALSE FLABEL%dWn", f_c_cnt);
    break;
case F_LE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOPLUS FLABEL%dWn", ++f_c_cnt);
    break;
case F_GE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOMINUS FLABEL%dWn", ++f_c_cnt);
    break;

```

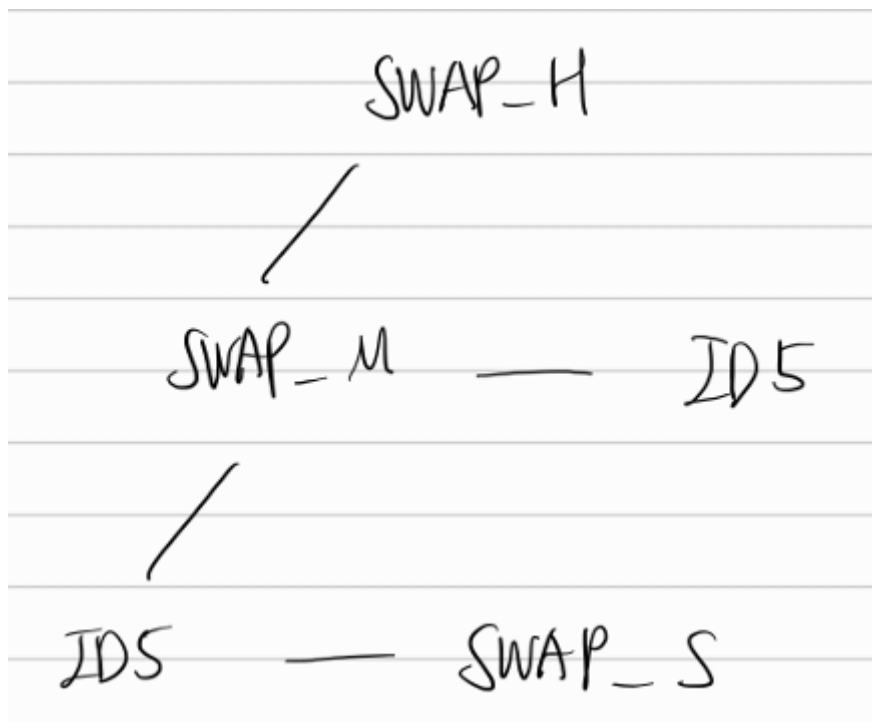
case FOR:

```
fprintf(fp, "GOTO FLOOP%d\\n", for_cnt - inner_f_cnt);  
fprintf(fp, "LABEL FLABEL%d\\n", f_c_cnt - inner_f_cnt);  
cnt--;  
break;
```

case FLOOPEND:

```
fprintf(fp, "LABEL FLOOP%d\\n", ++for_cnt);  
break;
```

4. swap 문:



- DFS 탐색 순서에 따른 실행 흐름:
SWAPS → ID5(첫 번째) → SWAPM → ID5(두 번째) → SWAPH → SWAPE
- swap문을 구현할 때 고민했던 점은 temp 변수의 처리였다. 두 값을 서로 교환하기 위해서는 임시 변수가 필요하므로, insertsym 함수를 사용하여 swap 문법 실행 시 TEMP라는 변수를 추가하고 DW로 메모리를 할당할 수 있도록 하였다.

- 여기서 ID5는 swap을 위한 새로운 토큰이며, SWAPS 노드에서 TEMP 변수를 생성하고, 각각 교환할 두 변수를 처리한 후 SWAPE에서 최종적으로 교환을 완료한다.

```

/* 위 트리를 규칙으로 만든 코드 */
swap_h      :      swap_m ID      {$2->token=ID5; $$=MakeOPTree(SWAPH, $1, $2);}

swap_m      :      ID WA swap_s    {$1->token=ID5; $$=MakeOPTree(SWAPM, $3, $2);}
              ;

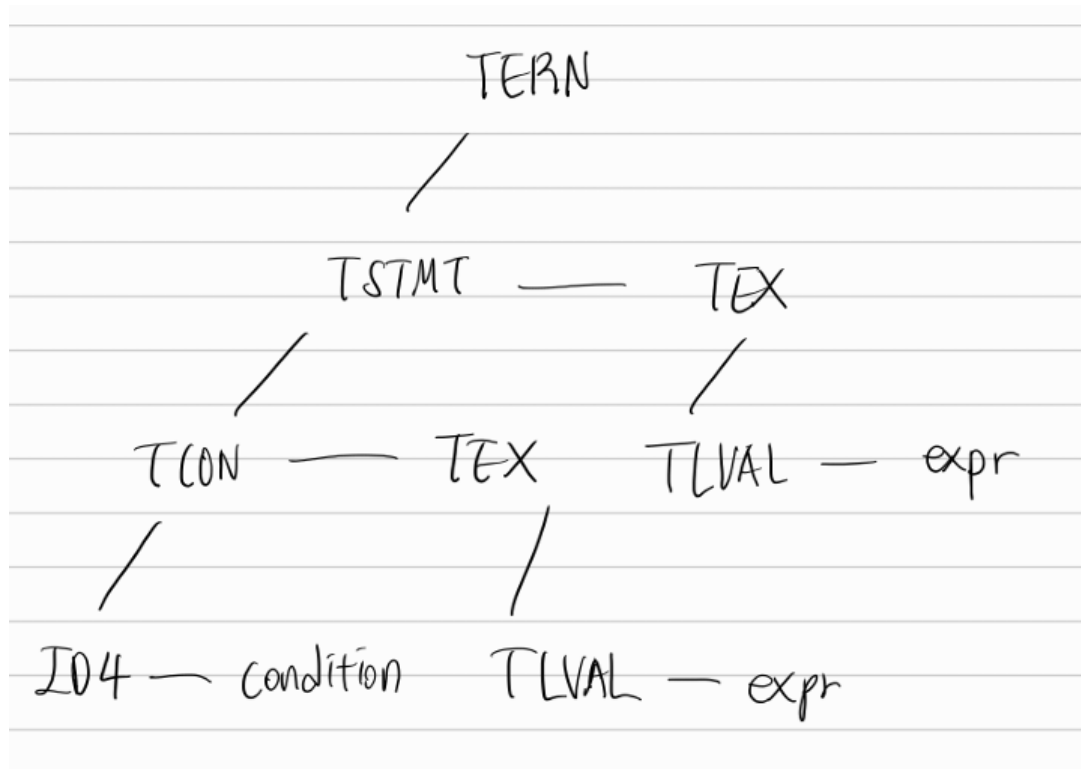
swap_s      :      {$$=MakeNode(SWAPS,SWAPS);}          ;

/* stmt 규칙 포함 */
swap_h SWAP  {$$=MakeOPTree(SWAPE, $1, NULL); }

/* 노드에 따라 코드 생성 */
case SWAPS:
    insertsym("TEMP");
    fprintf(fp, "LVALUE TEMP\n");
    break;
case SWAPE:
    fprintf(fp, "RVALUE TEMP\n");
    fprintf(fp, ":=\n");
    break;
case ID5:
    fprintf(fp, "RVALUE %s\n", symtbl[val]);
    fprintf(fp, ":=\n");
    fprintf(fp, "LVALUE %s\n", symtbl[val]);
    break;

```

5. ternary-operator (삼항연산자):



- 3항 연산자를 구현할 때 고민했던 점은 조건에 따라 특정 LVALUE에 다른 값을 할당해야 한다는 것이었다. 이를 변수명을 저장하는 방법으로 해결할 수 있다고 판단하였다. 설계한 트리는 위와 같다.
- DFS 탐색 순서에 따른 실행 흐름:
ID4 → condition → TCON → TLVAL → expr → TEX → TSTMT → TEX → TERN
- ID4 노드에서 변수명을 저장하고, TLVAL을 사용해서 저장된 변수명을 이용하여 조건에 따라 해당 변수에 값을 할당하는 코드를 생성한다. 저장 로직은 `char[50] str` 변수를 생성한 후 ID4에서 strcpy로 변수명을 저장하도록 구현하였다.
-

```

/* 위 트리를 규칙으로 만든 코드 */
t_stmt      :      t_con t_expr ELSE { $$=MakeOPTree(TSTMT, $1, $2); }
              ;

t_expr      :      t_lval expr THIS SAVE { $$=MakeOPTree(TEX, $1, $2); }
              ;

t_lval      :      { $$=MakeNode(TLVAL, TLVAL); }

```

```

;

t_con      :      ID ASSGN condition {$3->token=t_cond_check($3->token); $1-
>token=ID4; $$=MakeOPTree(TCON, $1, $3);}      ;

/* stmt 규칙 포함 */
t_stmt t_expr { $$=MakeOPTree(TERN, $1, $2); }

/* 노드에 따라 코드 생성 */
case T_EQ:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOTRUE TL%dWn", ++tl_cnt);
    break;
case T_NE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOFALSE TL%dWn", ++tl_cnt);
    break;
case T_LT:
    fprintf(fp, "-Wn");
    fprintf(fp, "COPYWn");
    fprintf(fp, "GOPLUS TL%dWn", ++tl_cnt);
    fprintf(fp, "GOFALSE TL%dWn", tl_cnt);
    break;
case T_GT:
    fprintf(fp, "-Wn");
    fprintf(fp, "COPYWn");
    fprintf(fp, "GOMINUS TL%dWn", ++tl_cnt);
    fprintf(fp, "GOFALSE TL%dWn", tl_cnt);
    break;
case T_LE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOPLUS TL%dWn", ++tl_cnt);
    break;
case T_GE:
    fprintf(fp, "-Wn");
    fprintf(fp, "GOMINUS TL%dWn", ++tl_cnt);
    break;

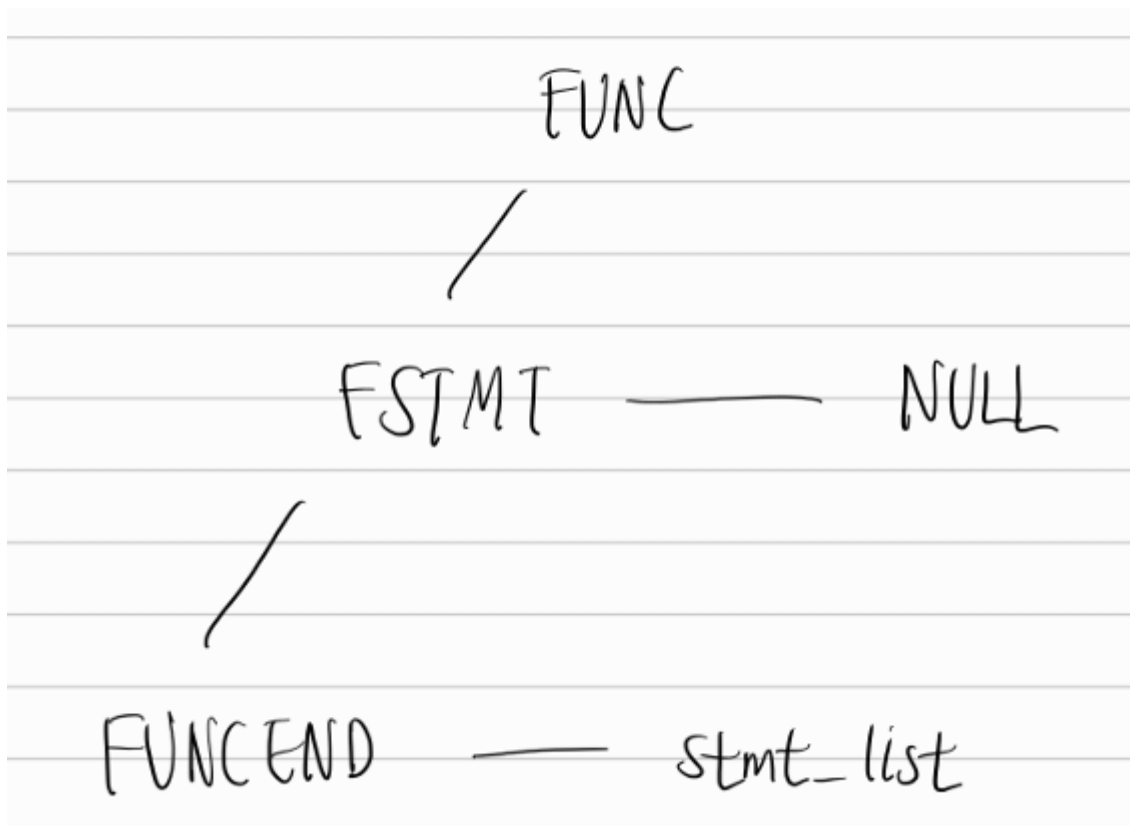
```

```

case TEX:
    fprintf(fp, ":=Wn");
    break;
case TLVAL:
    fprintf(fp, "LVALUE %sWn", str);
    break;
case TSTMT:
    fprintf(fp, "GOTO TOUT%dWn", tl_cnt);
    fprintf(fp, "LABEL TL%dWn", tl_cnt);
    break;
case TERN:
    fprintf(fp, "LABEL TOUT%dWn", tl_cnt);
    break;
case ID4:
    strcpy(str, symtbl[val]);
    break;

```

6. function:



- C언어에서 사용하는 함수는 함수의 선언과 호출로 구성된다. 이를 Stacksim에서 구현하려면 함수 선언 시에는 함수 내용을 실행하지 않고, 함수 호출 시에만 실행되어 호출 위치로 다시 돌아와야 한다. 따라서 이 언어에서 함수를 설계할 때 LABEL을 사용하도록 하였다.
- DFS 탐색 순서에 따른 실행 흐름:
FUNCEND → stmt_list → FSTMT → NULL → FUNC
- FUNCEND 노드에서 함수 끝으로 점프할 GOTO문과 CALLBACK LABEL을 생성하고, stmt_list를 처리한 후, FUNC 노드에서 함수의 끝을 의미하는 LABEL과 함수가 호출되었을 때 함수 내용을 모두 실행하고 다시 호출한 자리로 돌아갈 GOTO문을 생성한다.

```

/* 위 트리를 규칙으로 만든 코드 */
f_stmt_list      :      stmt_list f_end      {$$=MakeOPTree(FSTMT, $2, $1);}
;

f_end            :      { $$=MakeNode(FUNCEND, FUNCEND); }
;

/* stmt 규칙 포함 */
FUNC FNAME AND CONTENT LB f_stmt_list RB IN FN THIS INIT { $$=MakeOPTree(FUNC, $6,
NULL); }

/* CALL BACK NODE */
call_back        :      FNAME THIS CALL {$$=MakeNode(CALLBACK, CALLBACK);}
;

/* 노드에 따라 코드 생성 */
case FUNCEND:
    fprintf(fp, "GOTO FUNCLABEL%d\n", ++f_cnt);
    fprintf(fp, "LABEL CALLBACK%d\n", cb_cnt);
    cb_cnt--;
    break;
case FUNC:
    fprintf(fp, "GOTO FUNCOUT%d\n", ++fe_cnt);
    fprintf(fp, "LABEL FUNCLABEL%d\n", f_cnt--);
    f_cnt--;

```

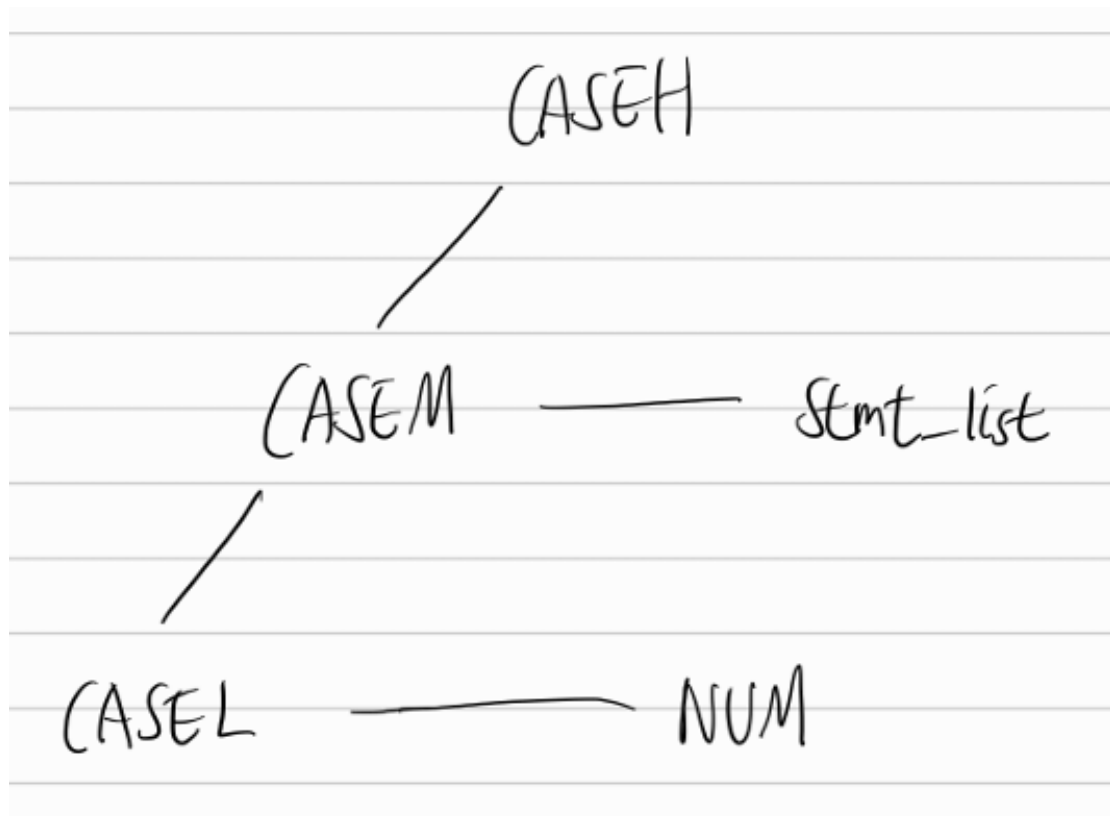
```

        break;
case CALLBACK:
    fprintf(fp, "GOTO CALLBACK%d\\n", cb_cnt+1);
    fprintf(fp, "LABEL FUNCOOUT%d\\n", fe_cnt);
    fe_cnt--;
    break;

```

- 이를 통해 함수 선언 시에는 함수 내용이 실행되지 않고, 함수 호출(CALLBACK) 시에만 해당 함수로 이동하여 실행 후 원래 위치로 복귀하는 정상적인 함수 메커니즘을 구현할 수 있었다.

7. switch 문:



- switch문은 내부에서 case문이 반복되어 나타나는 형태이다. 이를 구현하기 위해 MakeListTree 함수를 사용하여 case문이 반복되어 나타날 수 있는 case_list 규칙을 만들고 이를 switch문에서 사용하였다. 설계한 트리 구조는 위와 같다.
- DFS 탐색 순서에 따른 실행 흐름:

CASEL → NUM → CASEM → stmt_list → CASEH

- 트리를 설명하면 CASEL에서 switch문에서 입력받은 변수를 RVALUE로 한다. 그 후 NUM 값과 변수의 값이 같은 경우 stmt_list를 실행하고, 다를 경우 다음 case로 넘어가게 된다. MakeListTree를 통해 여러 case문이 연속적으로 처리될 수 있도록 구현하였다.

```
/* 위 트리를 규칙으로 만든 코드 */
case_h      :      case_m COLON stmt_list BREAK  {$$=MakeOPTree(CASEH, $1, $3);}
              ;

case_m      :      NUM CASE case_l              {$$=MakeOPTree(CASEM, $3, $1);} //todo
              ;

case_l      :      {$$=MakeNode(CASEL,CASEL);}
              ;

/* 위 규칙들로 만든 case_h */
case_list   :      case_list case_h              {$$=MakeListTree($1,
$2); }
              |      case_h                      {$$=MakeListTree(NULL,
$1); }
              ;

/* stmt 규칙 포함 */
switch_stmt :      SWITCH ID IS case_list {$2->token=ID4; $$=MakeOPTree(SWITCHH,
$2, $4);}
              ;

default_stmt :      DEFAULT COLON stmt_list BREAK {$$=MakeOPTree(DEFAULT, $3,
NULL); }
              ;

switch_stmt default_stmt { $$=MakeOPTree(SWITCH, $1, $2); }

/* 노드에 따라 코드 생성 */
case CASEL:
```

```

        fprintf(fp, "RVALUE %s\n", str);
        break;
case CASEM:
    fprintf(fp, "-\n");
    fprintf(fp, "GOTRUE CLABEL%d\n", ++case_cnt);
    break;
case CASEH:
    fprintf(fp, "GOTO SLABEL%d\n", s_cnt);
    fprintf(fp, "LABEL CLABEL%d\n", case_cnt);
    break;
case SWITCH:
    fprintf(fp, "LABEL SLABEL%d\n", s_cnt++);
    break;

```

8. printf & scanf:

```

/* stmt 규칙 포함 */
ID VAL SCANF {$1->token=ID2; $$=MakeOPTree(SCAN, $1, NULL);}
ID VAL PRINTF { $$=MakeOPTree(PRINT, $1, NULL); }

/* 노드에 따라 코드 생성 */
case SCAN:
    fprintf(fp, "INNUM\n");
    fprintf(fp, ":\n");
    break;
case PRINT:
    fprintf(fp, "OUTNUM\n");
    break;

```

- 입출력 함수인 scanf와 printf를 구현하기 위해 간단한 문법 규칙을 정의하였다.
- SCAN 노드에서는 INNUM으로 숫자를 입력받고 `:=` 연산자로 해당 변수에 값을 저장합니다. PRINT 노드에서는 OUTNUM으로 변수의 값을 출력한다. scanf에서 변수가 저장 대상(LVALUE)이 되어야 하기 때문에 ID 토큰을 ID2로 변경하였다.

코드 생성(Code Generation):

스택 기반 중간 코드 아키텍처.

스택 기반 명령어 세트 Example:

PUSH 10	// 상수 10을 스택에 푸시
RVALUE x	// 변수 x의 값을 스택에 푸시
LVALUE x	// 변수 x의 주소를 스택에 푸시
+	// 스택 상위 두 값을 팝하여 덧셈 후 결과 푸시
:=	// 할당: 값과 주소를 팝하여 메모리에 저장

코드 생성 함수 구조:

```
void prtcode(int token, int val) {
    switch (token) {
        case NUM:
            fprintf(fp, "PUSH %d\n", val);
            break;
        case ID:
            fprintf(fp, "RVALUE %s\n", symtbl[val]);
            break;
        case ADD:
            fprintf(fp, "+\n");
            break;
        // ... 기타 토큰들
    }
}
```

Examples:

단순 IF문으로 예시:

Cute Code:
시작할랭
x에 10을 저장할랭
만약에 x가 5보다 크다면 요기부터
x에 100을 저장할랭
요기까지 실행할랭

고마웁

생성 어셈블리:

```
LVALUE x
PUSH 10
:=
RVALUE x
PUSH 5
-
COPY
GOMINUS IFLABEL1
GOFALSE IFLABEL1
LVALUE x
PUSH 100
:=
LABEL IFLABEL1
HALT
$ -- END OF EXECUTION CODE AND START OF VAR DEFINITIONS --
DW x
END
```

While 반복문 코드 예시:

```
Cute Code:
시작할랭
    counter에 0을 저장할랭
    result에 1을 저장할랭

    counter가 5보다 작다면 요기부터
        result *= 2
        result의 값을 출력할랭
        counter++
    요기까지 반복할랭
고마웁
```

생성 어셈블리:

```

LVALUE counter
PUSH 0
:=
LVALUE result
PUSH 1
:=
LABEL WLOOP1
RVALUE counter
PUSH 5
-
COPY
GOPLUS WLABEL1
GOFALSE WLABEL1
LVALUE result
RVALUE result
PUSH 2
*
:=
RVALUE result
OUTNUM
LVALUE counter
RVALUE counter
PUSH 1
+
:=
GOTO WLOOP1
LABEL WLABEL1
HALT
$ -- END OF EXECUTION CODE AND START OF VAR DEFINITIONS --
DW counter
DW result
END

```

함수 선언 및 호출:

Cute Code:
시작할랭

함수이름은 func_a 이고 뭐가들었냐면 요기부터

a에 0을 저장할랭

일단은 i에 0을 저장할랭 i가 100보다 작다면 마지막에 i++ 하면서 요기부터

a++

요기까지 반복할랭

a의 값을 출력할랭

요기까지인 함수를 선언할랭

func_a를 호출할랭

a의 값을 출력할랭

고마웁

GOTO FUNCLABEL1

LABEL CALLBACK0

LVALUE a

PUSH 0

:=

LVALUE i

PUSH 0

:=

LABEL FLOOP1

RVALUE i

PUSH 100

-

COPY

GOPLUS FLABEL1

GOFALSE FLABEL1

LVALUE a

RVALUE a

PUSH 1

+

:=

LVALUE i

RVALUE i

PUSH 1

+

```

:=
GOTO FLOOP1
LABEL FLABEL1
RVALUE a
OUTNUM
GOTO FUNCOUT1
LABEL FUNCLABEL1
GOTO CALLBACK0
LABEL FUNCOUT1
RVALUE a
OUTNUM
HALT
$ -- END OF EXECUTION CODE AND START OF VAR DEFINITIONS --
DW a
DW i
END

```

라벨 관리 시스템.

라벨 카운터 관리:

```

int if_cnt = 0;      // IF문 라벨 카운터
int w_cnt = 0;       // WHILE문 라벨 카운터
int f_cnt = 0;       // 함수 라벨 카운터
int case_cnt = 0;    // CASE문 라벨 카운터
int tl_cnt = 0;      // 삼항연산자 라벨 카운터

```

중첩 구조 처리

```

int count_if_nodes(Node *n) {
    if (n == NULL) return 0;
    int count = 0;
    if (n->token == IF) count++;
    count += count_if_nodes(n->son);
    count += count_if_nodes(n->brother);
    return count;
}

```

라벨 할당 전략:

- 외부 IF문: IFLABEL1
- 내부 IF문: IFLABEL2
- 라벨 충돌 방지를 위한 카운터 관리

심볼 테이블 및 스코프 관리:

심볼 테이블 구조 및 구현.

```
#define MAXSYM 100
#define MAXSYMLEN 20

char symtbl[MAXSYM][MAXSYMLEN]; // 심볼명 저장 배열
int maxsym = 0;                  // 현재 등록된 심볼 개수
```

심볼 삽입 알고리즘.

```
int insertsym(char *s) {
    int i;
    // 기존 심볼 검색 (중복 방지)
    for(i = 0; i < maxsym; i++) {
        if(strcmp(s, symtbl[i]) == 0)
            return(i); // 기존 심볼 인덱스 반환
    }
    // 새 심볼 추가
    if (i < MAXSYM-1) {
        strcpy(symtbl[maxsym], s);
        maxsym++;
        return(maxsym-1);
    } else {
        printf("symbol table overflow\n");
    }
    return(0);
}
```

스코프 관리.

- 현재 구현에서는 Global Scope만 지원하지만, 향후 확장을 위한 설계:
- 스코프 스택 구조:
 - 함수 진입 시 새로운 스코프 생성
 - 블록 진입 시 중첩 스코프 생성
 - 변수 검색 시 현재 스코프부터 상위 스코프로 검색

변수 메모리 할당.

```
void dwgen() {
    fprintf(fp, "HALT\n");
    fprintf(fp, "$ -- END OF EXECUTION CODE AND START OF VAR DEFINITIONS --\n");
    // 모든 변수에 대해 메모리 공간 할당
    for(int i = 0; i < maxsym; i++)
        fprintf(fp, "DW %s\n", symtbl[i]);
    fprintf(fp, "END\n");
}
```

컴파일 과정 및 최적화

컴파일 과정 순서:

1. 어휘 분석기(Lexical Analyzer)
 - 소스코드를 토큰으로 분할
 - ex) "저장할랭" → SAVE 토큰
2. 구문 분석기(Parser)
 - 토큰들을 문법 규칙에 따라 파싱
 - AST(추상 구문 트리) 생성
3. 의미 분석기(Semantic Analyzer)
 - 타입 체크, 스코프 검사
 - ex) if_cond_check() 같은 의미 검사
4. 중간코드 생성기(Intermediate Code Generator)
 - AST를 중간 표현으로 변환
 - 제 프로젝트는 스택 기반 어셈블리 코드로
5. 코드 최적화기(Code Optimizer)

- 성능 향상을 위한 최적화
 - 제 프로젝트는 기본적인 최적화만 수행
6. 코드 생성기(Code Generator)
- 최종 타겟 코드 생성
 - a.asm 파일 도출

※ 심볼테이블 매니저와 오류처리는 모든 단계에서 사용됨

상세 설명:

어휘 분석기(Lexical Analyzer).

역할: 소스코드를 토큰으로 분할 구현: Flex(Lex) 도구 사용

e.g)

```
// 어휘 분석 예시
"저장할랭"    → SAVE 토큰
"만약에"      → IF 토큰
"요기부터"    → LB 토큰
{id}          → ID 토큰 + 심볼테이블 등록
{digit}+      → NUM 토큰
```

심볼 테이블과의 상호작용:

```
{id} {
    temp = insertsym(yytext);    // 심볼테이블에 등록
    yylval = MakeNode(ID, temp); // AST 노드 생성
    return(ID);
}
```

구문 분석기(Parser).

역할: 토큰을 문법 규칙에 따라 파싱하여 AST 생성 구현: Bison(Yacc) 도구 사용

e.g)

```
// 구문 분석 규칙 예시
stmt : ID ASSGN expr THIS SAVE {
    $1->token = ID2;           // 의미 액션
```

```
$ = MakeOPTree(ASSGN, $1, $3);    // AST 노드 생성
}
```

에러 처리:

```
stmt_list : stmt_list stmt {$=MakeListTree($1, $2);}
          | stmt           {$=MakeListTree(NULL, $1);}
          | error           { errorcnt++; yyerrok; } // 에러 복구
```

의미 분석기(Semantic Analyzer).

역할: 타입 체크, 스코프 검사, 의미적 오류 검출 등

e.g)

```
// 조건문 타입 변환
int if_cond_check(int token) {
    if (token == EQ) return IF_EQ;    // 의미 분석을 통한 토큰 변환
    if (token == LT) return IF_LT;
    // ... 기타 조건 연산자들
}
```

※ 앞서 서술하였지만 현재는 전역 스코프(Global Scope)만 지원.

중간코드 생성(Intermediate Code Generator).

역할: AST를 스택 기반 중간코드로 변환

e.g)

```
void codegen(Node* root) {
    DFSTree(root);    // 깊이 우선 탐색으로 AST 순회
}
```

중간코드 형태:

```
PUSH 10    // 상수
RVALUE x    // 변수 값
+          // 연산
```

```
LVALUE temp // 변수 주소
:=          // 할당
```

코드 최적화기(Code Optimizer).

이 프로젝트에 구현된 최적화:

- 기본적인 상수 전파, 불필요한 라벨 제거, 조건부 점프 최적화

코드 생성기(Code Generator).

역할: 최종 어셈블리 코드 생성

e.g)

```
void prtcode(int token, int val) {
    switch (token) {
        case ADD:
            fprintf(fp, "+%dn");
            break;
        case ASSGN:
            fprintf(fp, ":%dn");
            break;
        // ... 기타 토큰들
    }
}
```

> a.asm 도출.

III. 결과 및 실행화면

arithmetic:

```
시작할래  
a에 15를 저장할래  
b에 4를 저장할래  
c에 0을 저장할래  
d에 1을 저장할래  
  
c에 a 더하기 b를 저장할래  
  
a++  
b--  
c += 10  
d *= 5  
  
a의 값을 출력할래  
b의 값을 출력할래  
c의 값을 출력할래  
d의 값을 출력할래  
고마워
```

```
Start of execution.  
164295  
Successfully executed.
```

```
[DATA Dump]  
Loc#  Symbol      Value  
  0    a           16  
  1    b            4  
  2    c           29  
  3    d            5  
[End of Dump]
```

if-else:

```
시작할랭
x에 15를 저장할랭
y에 20을 저장할랭

만약에 x가 10보다 크다면 요기부터
    만약에 y가 15보다 크다면 요기부터
        result에 100을 저장할랭
    요기까지 실행할랭
    아니라면 요기부터
        result에 50을 저장할랭
    요기까지 실행할랭
    요기까지 실행할랭

result의 값을 출력할랭
고마워
```

Successfully assembled.

Start of execution.

100

Successfully executed.

[DATA Dump]

Loc#	Symbol	Value
0	x	15
1	y	20
2	result	100

[End of Dump]

while:

```
시작할랭
counter에 0을 저장할랭
result에 1을 저장할랭

counter가 5보다 작다면 요기부터
    result *= 2
    result의 값을 출력할랭
    counter++
    요기까지 반복할랭
고마워
```

Successfully assembled.

Start of execution.

2481632

Successfully executed.

[DATA Dump]

Loc#	Symbol	Value
0	counter	5
1	result	32

[End of Dump]

for:

시작할래

sum에 0을 저장할래

일단은 i에 1을 저장할래 i가 5보다 작거나 같다면 마지막에 i++ 하면서 요기부터

sum += i

sum의 값을 출력할래

요기까지 반복할래

고마워

Successfully assembled.

Start of execution.

1361015

Successfully executed.

[DATA Dump]

Loc#	Symbol	Value
0	sum	15
1	i	6

[End of Dump]

ternary-operator:

```
시작할랭
  a에 10을 저장할랭
  b에 20을 저장할랭
  max에 a가 b 보다 크다면 a를 저장할랭 아니라면 b를 저장할랭
고마웁
```

```
Successfully assembled.

Start of execution.

Successfully executed.

[DATA Dump]
Loc#  Symbol      Value
  0    a           10
  1    b           20
  2   max          20
[End of Dump]
```

printf & scanf

```
시작할랭
  num의 값을 입력받아칭
  num의 값을 출력할랭
고마웁
```

```
Successfully assembled.

Start of execution.
1000
1000
Successfully executed.

[DATA Dump]
Loc#  Symbol      Value
  0   num          1000
[End of Dump]
```

'1000'을 입력하였다.

switch:

```
시작할랭
    day에 7을 저장할랭

    여러 경우가 있는 day가
        1일 경우엔: result에 100을 저장할랭
        빠져나가봐
        7일 경우엔: result에 200을 저장할랭
        빠져나가봐
        다 아닌경우엔: result에 50을 저장할랭
        빠져나가봐

    result의 값을 출력할랭
고마웜
```

```
Successfully assembled.

Start of execution.
200
Successfully executed.

[DATA Dump]
Loc#  Symbol      Value
   0   day         7
   1  result      200
[End of Dump]
```

function:

```
시작할랭
    함수이름은 func_a 이고 뭐가들었냐면 요기부터
        a에 0을 저장할랭
        일단은 i에 0을 저장할랭 i가 100보다 작다면 마지막에 i++ 하면서 요기부터
            a++
        요기까지 반복할랭
        a의 값을 출력할랭
    요기까지인 함수를 선언할랭

    func_a를 호출할랭

    a의 값을 출력할랭
고마웜
```

```

Successfully assembled.

Start of execution.
100100
Successfully executed.

[DATA Dump]
Loc#  Symbol      Value
   0   a           100
   1   i           100
[End of Dump]

```

swap:

```

시작할래
  x에 10을 저장할래
  y에 20을 저장할래

  x와 y의 값을 서로 바꿔줄래
고마워

```

```

Successfully assembled.

Start of execution.

Successfully executed.

[DATA Dump]
Loc#  Symbol      Value
   0   x           20
   1   y           10
   2  TEMP          10
[End of Dump]

```

IV. 토론 및 결론

본 프로젝트는 제가 사회에 관련한 뉴스와 여러 매체들을 보면서 날카로워진 한국 사회를 이 컴파일러 프로젝트를 통하여 "고마왕", "요기부터", "저장할랭" 등의 귀여운 한국어 표현을 통해 프로그래밍의 진입 장벽을 낮추고 둥글둥글한 사회를 만들어보려는 매우 의미 있는 접근이었습니다. 특히 익숙해지지 않으면 이해하기 힘든 영어 중심의 프로그래밍 환경에서 모국어를 활용한 코딩 경험을 제공함으로써, 초보자들도 코딩하기 쉽도록 구현해보는 것도 목표 중에 하나였습니다. 그리고, 평소 C 언어로 코딩을 하면서 불편했던 여러가지 항목들(ex: python과 다르게 끝에마다 ;을 붙여야하는 등)도 개선해보고자 하였던 시도도 의미가 있는 것 같아 좋았습니다.

프로젝트 초기에는 컴파일러라는 복잡한 시스템을 구현한다는 것 자체가 막막하게 느껴졌습니다. 실습시간에 열심히 하였음에도 불구하고 처음해보는 기능을 구현하려 했기 때문에 Flex와 Bison의 문법(.l, .y 파일들을 작성하는), AST 구조 설계, 코드 생성 과정 등 모든 것이 낯설고 어려웠습니다. 특히 한국어 문법을 BNF 형태로 표현하고 이를 파서로 구현하는 과정에서는 며칠동안 고민하고 과제를 수행하면서 헤맸던 경험이 다수 있습니다.

하지만 몇날 며칠 꾸준히 프로젝트에 매달리면서 점진적으로 이해의 폭이 넓어지는 것을 느낄 수 있었습니다. 처음에는 단순한 토큰 인식조차 어려웠지만, 매일 조금씩 기능을 추가하고 디버깅하는 과정을 반복하면서 컴파일러의 전체적인 동작 원리가 점점 학습되는 느낌이 들었습니다.

가장 큰 도전은 한국어 문법의 특성을 프로그래밍 언어 문법으로 자연스럽게 변환하는 것이었습니다. 예를 들어 "만약에 x가 y보다 크다면"과 같은 한국어의 어순과 조사 사용을 BNF 문법으로 표현하면서도 파싱이 용이하도록 설계하는 과정에서 많은 시행착오가 있었습니다. 또한 while문과 for문에서 루프 시작점 라벨 배치를 위해 추가 노드들을 도입하는 것을 떠올린 것은 획기적으로 느껴졌습니다. swap 기능에서도 C언어에서 3줄 이상의 코드가 필요한 것과 달리 임시 변수를 자동으로 생성하고 코드 한 줄로 수행할 수 있도록 하는 메커니즘도 매우 마음에 들었습니다.

처음에는 각 기능을 어떻게 구현할지도 모르고 막막하였지만, 몇날 며칠동안의 지속적인 학습과 실험을 통해 하나씩 극복해 나가면서 자신감이 생겼습니다. 이 프로젝트를 하면서 컴파일러 강의 자료는 수도 없이 보았고, 모르는 정보는 계속 탐구하면서 극복해낼 수 있었습니다. 지금까지 이렇게 긴 기간동안 과제에 매달려본 기억이 없었는데, 좋은 기회였다고 생각합니다.

Cute Compiler의 아이디어는 최근 날카로운 한국 사회를 둥글둥글하게 해보자는 의미에서 시작하였는데, 이는 최근 사회 이슈에 관해 제가 배우는 분야를 접목시켜 해결을 해보려는 좋은 접근이었다고 생각합니다. 비록 이 언어의 컴파일러는 과제 그치는 것 같지만, 만약 이것을 우리 분야에서 좀 알려져 누구 한 명이라도 둥글둥글한 언어에 익숙해질 수만 있다면 "이 프로젝트는 매우 성공적인 프로젝트다."라고 할 수 있을 것 같습니다.

지금 이렇게 기본 기능만 구현하는 것도 몇날 며칠 장기간 고민하면서 겨우 구현하였는데, 여러 프로그래밍 언어를 개발한 사람들이 정말 대단하고 똑똑한 사람들이라는 것을 새삼 느끼게 되었습니다.